

ROADMAP

Roadmap

- v0.1.0 — Production primitives (released)
- v0.2.0 — Tool use protocol + scale
- v0.3.0 — Multi-agent patterns
- v0.4.0 — Embeddability
- v1.0.0 — Stability and adoption
- Stretch goals (no committed timeline)
- How to influence priorities

Roadmap

What marrow is working toward, in priority order. Dates are intent; OSS schedules are aspirational.

v0.1.0 — Production primitives (released)

Production primitives, observability hooks, persistence, distribution-ready packaging.

- ☒ Per-call timeouts on `Provider::generate / generate_stream`
 - ☒ `CancelToken` cooperative cancellation
 - ☒ Bounded `AgentRouter` inboxes (`Reject / DropOldest / DropNewest`)
 - ☒ `Engine::shutdown()` graceful stop signal
 - ☒ Message size cap (4 MiB)
 - ☒ `RetryPolicy` with exponential backoff + jitter
 - ☒ `RateLimiter` token bucket
 - ☒ `StateStore` interface + `InMemoryStateStore` + `SQLiteStateStore`
 - ☒ `TraceSink` interface + `NullTraceSink / PrintTraceSink / OpenTelemetryTraceSink`
 - ☒ `UsageTracker` for token + cost accounting
 - ☒ Structured logging via `marrow.logging_config`
 - ☒ `STABILITY.md` with semver + deprecation policy
 - ☒ `BENCHMARKS.md` with micro + e2e + `LangGraph` comparison
 - ☒ Docs site (mkdocs-material) with autogenerated API reference
 - ☒ PyPI publishing workflow (trusted publishing)
-

v0.2.0 — Tool use protocol + scale

Full tool-use loop, larger-scale tests, more providers.

- ☐ Native multi-turn tool-use protocol — `tool_use` blocks in / `tool_result` blocks out, integrated with `Agent.step()`
- ☐ OpenAI tool calling end-to-end
- ☐ Anthropic tool use end-to-end
- ☐ Structured tool errors (typed exceptions, not just strings)
- ☐ Cost ceilings — `RetryPolicy` aware of budget;
`Runtime(max_cost_usd=...)`
- ☐ Native streaming-with-cancellation (cooperative cancel checked between chunks)
- ☐ Provider plugin system — third-party providers as `marrow-provider-*` packages
- ☐ Soak test: 10,000-agent stress test in CI nightly
- ☐ Property tests with hypothesis for router + cache invariants

- ☐ ThreadSanitizer fail-on-warning in CI (currently informational)
-

v0.3.0 — Multi-agent patterns

The patterns most real agent systems converge on, as first-class building blocks.

- ☐ SupervisorPattern — one agent routes work to others
 - ☐ ReflectionPattern — agent critiques its own output before returning
 - ☐ VotingPattern — N agents answer the same prompt; aggregate
 - ☐ ReActPattern — built-in tool-using loop with bounded steps
 - ☐ DAG executor — execute a Graph with parallel branches where independent
 - ☐ Shared memory across agents (RAG primitives)
 - ☐ Async-iterator streaming API at the SDK level (async for token in agent.astream())
-

v0.4.0 — Embeddability

The C++ core as a real product for non-Python downstream users.

- ☐ C++ shared library build (libmarrow.so / .dylib / .dll)
 - ☐ Plain C ABI for cross-language embedding
 - ☐ C++ plugin loader for native providers (dlopen .so providers)
 - ☐ llama.cpp provider in C++
 - ☐ Go bindings (cgo wrapper)
 - ☐ Rust bindings (marrow-rs crate)
-

v1.0.0 — Stability and adoption

- ☐ ABI stability promise per module
 - ☐ At least 3 unrelated production users
 - ☐ Bus factor ≥ 2 committers
 - ☐ Published on PyPI for ≥ 6 months
 - ☐ Documented migration paths from LangGraph, CrewAI, AutoGen
 - ☐ Real-world benchmarks (not just microbench) with reproducible scripts
-

Stretch goals (no committed timeline)

- WebAssembly target
 - TypeScript bindings via WASM
 - Distributed routing (one Engine per host, gossip protocol)
 - Built-in vector store with HNSW
 - Tracing UI similar to LangSmith but local-first
-

How to influence priorities

Open an issue with the roadmap label arguing for what should move up or down. The roadmap is not a contract — it's a published intention. If your use case needs something that isn't on it, say so and we'll have the conversation.